CS611 MACHINE LEARNING ENGINEERING · ASSIGNMENT 1

Data Pipeline: from raw CSVs to a to a loan-default feature store

How four raw files become ML-ready bronze, silver and gold tables: the design choices, the data-quality fixes, and the controls that keep future information out of the model.

Bronze · land **Silver** · clean **Gold** · serve

Thai Hoang Quang Hung
September 2026

EXECUTIVE SUMMARY

Four raw files become one label store and one feature store, ready for ready for model training

12,500

loans labelled, one per customer, started Jan 2023 to Jan 2025

28.8%

default rate: 30+ days past due at month 6 of the loan

59

model features on the application date, including 18 engineered ones; no personal identifiers

The question

When a customer applies for a loan, how likely are they to fall 30 or more days behind on repayments within six months?

What was built

A PySpark pipeline in Docker that follows the Medallion Architecture: bronze lands the raw data, silver cleans it, gold serves it to the model.

Why it can be trusted

Features use only what was known on the application date, labels come from repayments six months later, and one command rebuilds every table.

THE RAW DATA

Four raw files, three grains, one join key

File	What it holds	Grain	Rows	Period	Role
lms_loan_daily.csv	Monthly repayment record per loan: amount due, paid, overdue, balance	loan × month on book (0 to 10)	137,500	Jan 2023 – Nov 2025	Label
features_attributes.csv	Age and occupation, plus name and SSN (personal identifiers)	customer, on application date	12,500	Jan 2023 – Jan 2025	Features
features_financials.csv	Income, debts, cards, interest rate, repayment behaviour, credit history	customer, on application date	12,500	Jan 2023 – Jan 2025	Features
feature_clickstream.csv	20 anonymised web-activity signals (fe_1 to fe_20)	customer × month	215,376	Jan 2023 – Dec 2024	Features

Key finding: each customer has exactly one loan, and their attributes and financials are dated on the loan start date (12,500 of 12,500), so the application date is the join key.

Source: raw files provided for the assignment; counts from the pipeline's bronze layer

EXPLORATORY DATA ANALYSIS

Credit behaviour, not loan terms, drives default

`lms_loan_daily`

Every loan has the same terms

$10,000

amount of every loan

10 months

tenure of every loan

~500

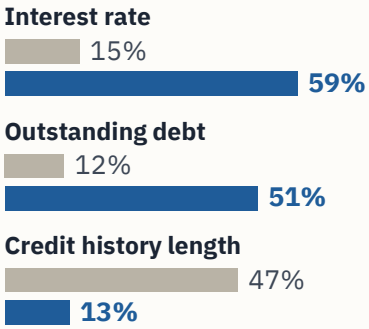
new loans a month (454 to 543)

→ Terms carry no signal, so this file supplies the label only.

`features_financials`

Credit behaviour splits risk sharply

Default rate, **lowest** vs **highest** fifth of customers

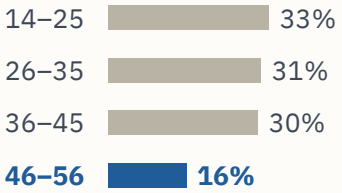


→ All fields kept, plus debt- and EMI-to-income ratios in gold.

`features_attributes`

Age matters, occupation barely does

Default rate by age band



→ Keep Age (319 impossible values nulled); drop Name and SSN.

`feature_clickstream`

20 signals, weak and independent

8,974 × 24

customers × months, a complete panel

0.01

largest correlation between two signals

0.12

strongest link of any one signal to default

→ Weak alone; only the application-month row is used (slide 9).

Default rates join each customer's cleaned features to their month-6 label (12,500 loans); clickstream links are Spearman correlations

MEDALLION ARCHITECTURE

One job per layer: land, clean, serve

Raw
source files

```
lms_loan_daily
feature_clickstream
features_attributes
features_financials
```

Four CSV files in data/. The source of truth; the pipeline never modifies them.

Bronze
land it

```
lms_loan_daily
feature_clickstream
features_attributes
features_financials
```

One CSV per month holding that month's rows, as the source system would deliver them. Values unchanged.

Silver
clean it

```
lms_loan_daily
features_clickstream
features_attributes
features_financials
```

One Parquet per month: schema enforced, dirty values repaired, days past due derived.

Gold
serve it

```
feature_store
label_store
```

One Parquet per month: applicants' features, and each loan's month-6 outcome.

Built like a production job: main.py loops over monthly snapshot dates and every layer writes one file per month, so any month can be re-run on its own without touching the raw files.

Code: main.py · utils/data_processing_bronze.py, _silver.py, _gold.py · output in datamart/bronze, silver, gold

SILVER LAYER

Silver enforces types and repairs the common dirty values

Problem found in EDA	Raw example	Values	Silver rule
Stray characters in numbers	52312.68_ · 40_ · 26_	2,009	Strip "_" (and "-" where a sign is junk), then cast
Impossible ages	−500 · 8,678	319	Null outside 14 to 100 years, not guessed
Placeholder values	Occupation "_______" · credit limit "_" · "NM"	2,572	"Unspecified" · 0 · "No"
Numbers hidden in text	"10 Years and 9 Months"	all rows	Credit_History_Age_Months = 129
Loose types	dates and amounts as text	all columns	Explicit schema per table: dates, integers, Decimal(10,2); loans gain mob and dpd

⚠ **Known gaps, left for the next iteration:** Credit_Mix "_" (2,611) and Payment_Behaviour "!@9#%8" (998) stay as their own categories; outliers such as 1,756 bank accounts or 5,789% interest are not capped. Tree models tolerate both, but silver should null them.

Counts on the full raw files; rules in utils/data_processing_silver.py. Name and SSN stay in silver and are dropped in gold.

LABEL DEFINITION

Default = 30+ days overdue at month 6

1 · Count missed instalments

Each month the loan system records the amount due and the amount overdue. Missed instalments = overdue ÷ monthly instalment, rounded up.

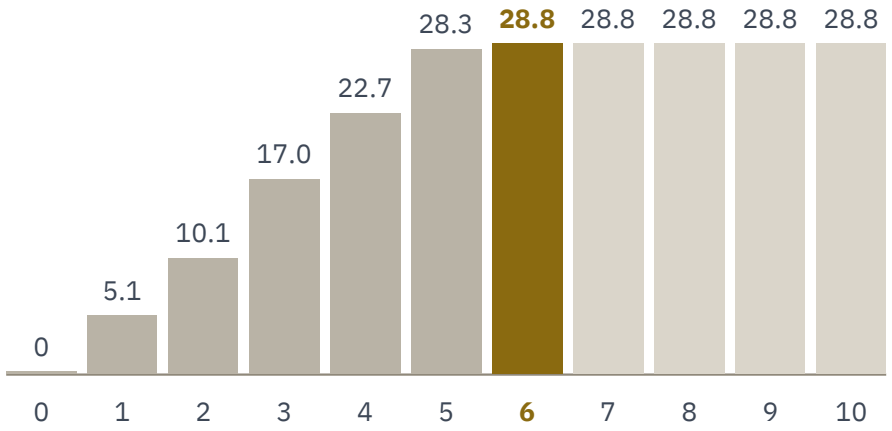
2 · Convert to days past due

Step back that many months to the first missed date; DPD = days from then to the snapshot date (the Lab 2 method, in silver).

3 · Read the label at month 6

Gold keeps each loan's month-6 row: label = 1 if DPD ≥ 30, else 0, stored with label_def = 30dpd_6mob.

Share of loans 30+ days past due, by month on book (%)



Why month 6: the rate reaches its final 28.8% at month 6 and stays flat, so it is the earliest point that catches every default.

Source: silver lms_loan_daily, 12,500 loans with 10-month tenure; label logic in utils/data_processing_gold.py

GOLD LAYER

One label store, one feature store, joined 1:1

`label_store`
12,500 rows · loan_id, loan_start_date, label, label_def, snapshot_date (month 6)

1 : 1 join
Customer_ID + application date

`feature_store`
12,500 rows · one per applicant · 2 keys + 59 features

Feature group	Columns	Count
Profile, income, cash flow	Age, Occupation, Annual_Income, Monthly_Inhand_Salary, Monthly_Balance, Amount_invested_monthly, Total_EMI_per_month	7
Credit in use	Num_Bank_Accounts, Num_Credit_Card, Num_of_Loan, Interest_Rate, Outstanding_Debt, Credit_Utilization_Ratio, Changed_Credit_Limit	7
Repayment and history	Delay_from_due_date, Num_of_Delayed_Payment, Num_Credit_Inquiries, Credit_Mix, Payment_of_Min_Amount, Payment_Behaviour, Credit_History_Age_Months	7
Engineered in gold	n_<loan type> × 9, Total_Loans, n_secured, n_unsecured, n_distinct_loan_types, debt_to_income, emi_to_income, Spend_Level, Payment_Value, Credit_Mix_MinAmt	18
Clickstream	fe_1 to fe_20 from the application month only; empty for the 3,526 loans from Jul 2024	20

Parquet per month in datamart/gold; feature snapshot_date = application date = label loan_start_date

LEAKAGE CONTROLS

Every feature is fixed on the application date, so the model the future future

MONTH 0 · APPLICATION

Features: attributes, financials and that month's clickstream

MONTHS 1 TO 5 · REPAYMENTS

Observed by the loan system, but never used as features

MONTH 6 · LABEL

Is the loan 30+ days past due?
Yes = 1, No = 0

One month per partition

Gold reads only the application month's attributes and financials, dated on the loan start for 12,500 of 12,500 customers.

Repayments stay in the label

Paid, overdue and balance amounts from the loan system never enter the feature store.

Clickstream from the same month

Joined on customer and application month. The 61% of clickstream rows dated after a loan start are never used.

Nothing fitted in the pipeline

No imputation, scaling or encoding: engineered ratios use one row each; anything learned is fitted after the split.

Checks run on the datamart: snapshot dates, clickstream dates versus each customer's loan start